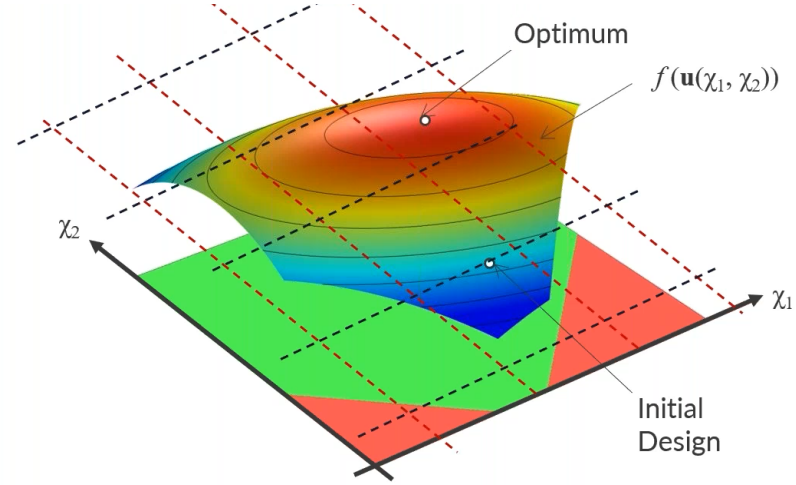
ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Τμήμα Μηχανολόγων Μηχανικών

ΕΡΓΑΣΙΑ ΒΕΛΤΙΣΤΟΠΟΙΗΣΗΣ

Βέλτιστη κατανομή οριακού στρώματος μέσω μηχανισμού έγχυσης για ελαχιστοποίηση τριβών σε πλάκα



Μάθημα: Μέθοδοι Βελτιστοποίησης
Φοιτητής: Marios Petropoulos
Α.Μ.: mc22043

Ακαδημαϊκό Έτος 2025–2026

Εισαγωγή

Στόχος της παρούσας εργασίας είναι η ελαχιστοποίηση των τριβών σε τοίχωμα όπου ρέει ρευστό μέσω μηχανισμού έγχυσης κάθετο στη ροή, ο οποίος έχει τη δυνατότητα να αλλάζει την κατανομή και το πάχος του οριακού στρώματος της ροής. Η βελτιστοποίηση γίνεται μέσω των 3 μεταβλητών σχεδιασμού (design variables) a,b,c οι οποίες τροποποιούν τον τρόπο με τον οποίο γίνεται η έγχυση από τον μηχανισμό.

Πριν από την διαδικασία βελτιστοποίησης γίνεται ανάπτυξη κώδικα για τον υπολογισμό των παραγώγων ευαισθησίας της αντικειμενικής συνάρτησης, με τους διάφορους τρόπους που έχουμε διδαχθεί στο μάθημα (FD, DD, Adjoint Methods). Η αναζήτηση του βέλτιστου σετ τιμών των a,b,c γίνεται με την συνεχή συζυγή μέθοδο (Continuous Adjoint).

Ερώτημα 1: Λογισμικό Υπολογισμού Αντικειμενικής Συνάρτησης

Η πλάκα, πάνω από την οποία ρέει το ρευστό, έχει μήκος x=1. Ο μηχανισμός έγχυσης δε, είναι τοποθετημένος στο τμήμα της πλάκας 0.2< x <0.8. Ο τύπος που δίνει την ταχύτητα έγχυσης κατά μήκος της πλάκας είναι ο παρακάτω:

$$v_0(x) = -\left(ax^2 + bx + c \cdot \sin\left(\frac{\pi x}{6}\right)\right)^2$$

Επιπλέον, γνωρίζουμε πως η παροχή που διέρχεται από τις οπές έγχυσης πρέπει να είναι σταθερή και ίση με -0.05U. Αυτό, μεταφράζεται στην σχέση:

$$\int_{0.2}^{0.8} v_0(x)\, dx = -0.05\, U$$

Εδώ να αναφερθεί πως μέχρι το στάδιο της βελτιστοποίησης, αυτός ο περιορισμός δεν λήφθηκε άμεσα υπόψην, παρά μόνον έμμεσα μέσω της επιλογής του σετ των design variables για το οποίο έγιναν οι υπολογισμοί των παραγώγων. Συγκεκριμένα, η επιλογή του σετ είναι η εξής:

$$\boldsymbol{b} = \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -0.6921 \end{bmatrix}$$

Ο νόμος που ακολουθεί η ταχύτητα έγχυσης προφανώς επηρεάζει το πάχος του οριακού στρώματος, για αυτό και λαμβάνεται υπόψην στην διαφορική εξίσωση υπολογισμού του, η οποία είναι δοσμένη από την εκφώνηση και φαίνεται παρακάτω:

$$\left(\frac{2}{\pi} - \frac{1}{2}\right) \frac{U}{\nu}\, \delta\, \frac{d\delta}{dx} \;+\; \frac{v_0(x)}{\nu}\, \delta \;=\; \frac{\pi}{2}$$

όπου $U = 5.33$ και $\nu = \frac{\mu}{\rho}$ με $\mu = 1.75 \times 10^{-5}$, $\rho = 1.25$ όλα σε SI units και γνωστά δεδομένα του προβλήματος με βάση τον αριθμό μητρώου (mc22043).

Η ΔΕ αυτή, αποτελεί την Primal εξίσωση του προβλήματος και η επίλυση της γίνεται με Runge-Kutta 2ης τάξης με αρχική συνθήκη $\delta(0) = 10^{-6}$. Η κατανομή του πάχους του οριακού στρώματος φαίνεται στο παρακάτω σχήμα:

Σχήμα 1: Πάχος οριακού στρώματος

Μετά την εύρεση του πάχους του οριακού στρώματος περνάμε στον υπολογισμό της αντικειμενικής συνάρτησης F, της δύναμης αντίστασης δηλαδή, που δίνεται από τον τύπο:

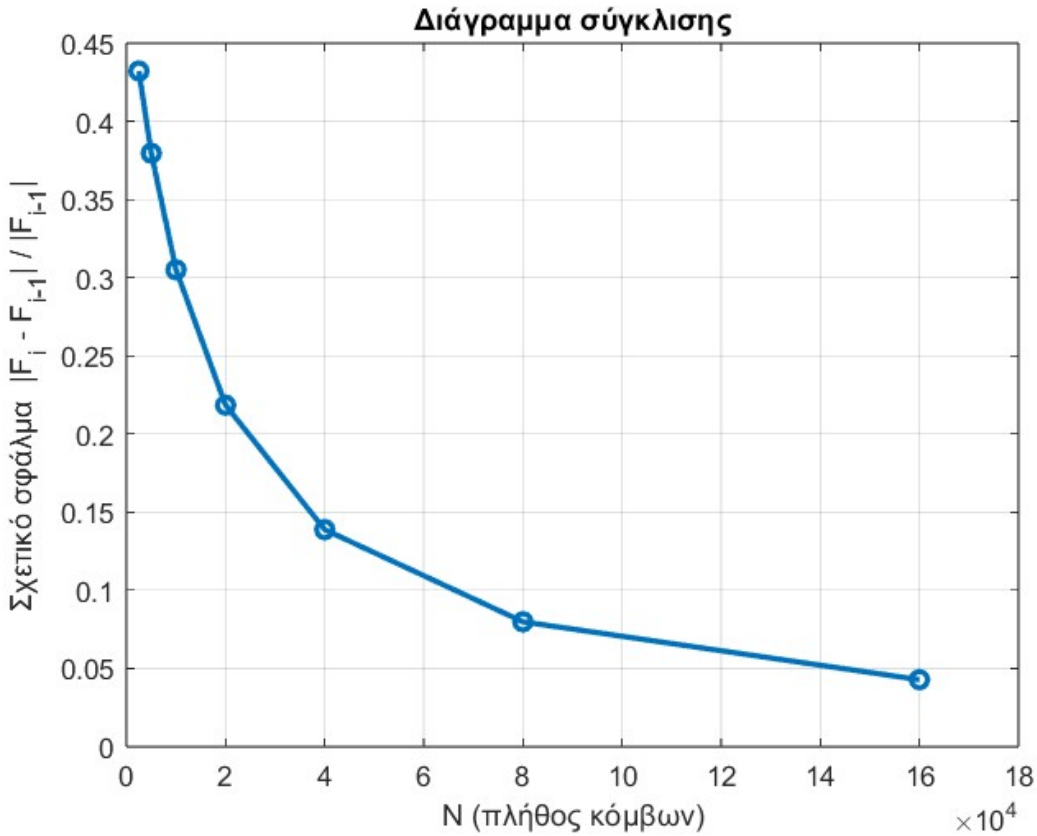
$$F = \int_0^l \tau_w \, dx$$

όπου

$$\tau_w(x) = \frac{\pi}{2} \frac{\mu U}{\delta}$$

Στο σημείο αυτό, γίνεται διερεύνυση ανεξαρτησίας ανεξαρτησίας της υπολογιζόμενης τιμής της F ως προς το πλήθος ισαπεχόντων κόμβων επί της πλάκας ξεκινώντας από N=1250 και διπλασιάζοντας κάθε φορά τον αριθμό των κόμβων,

έως ότου το σχετικό σφάλμα της F συγκλίνει κάτω από μία συγκεκριμένη τιμή (πχ 0.1). Παρακάτω φαίνεται το διάγραμμα σύγκλισης της F:



Σχήμα 2: Σύγκλιση της F

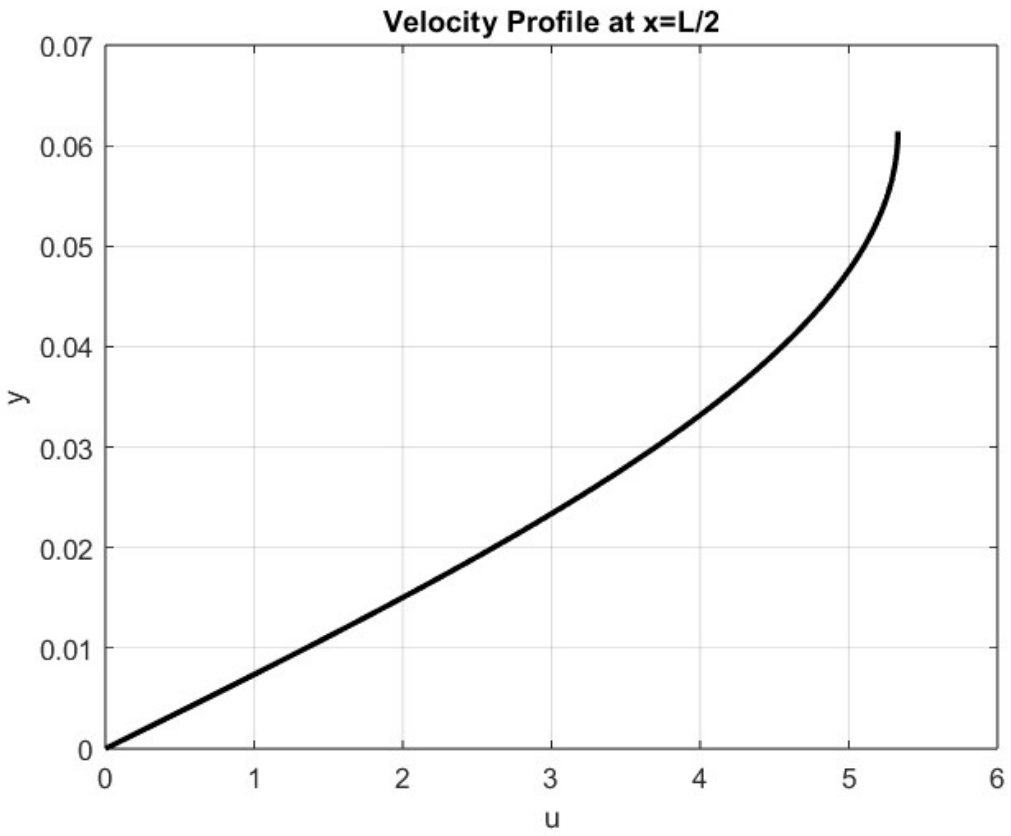
Παρατηρούμε πως η F συγκλίνει για N = 80001 κόμβους. Συνεπώς, κρατάμε αυτό το μέγεθος για όλα τα ερωτήματα. Για το σετ των a,b,c που επιλέχθηκε, με τη μέθοδο τραπεζίου υπολογίζεται:

$$\boxed{F = 2.262682455006 \times 10^{-2}}$$

Επίσης, είναι πια γνωστό και το profile της ταχύτητας κατά μήκος της πλάκας:

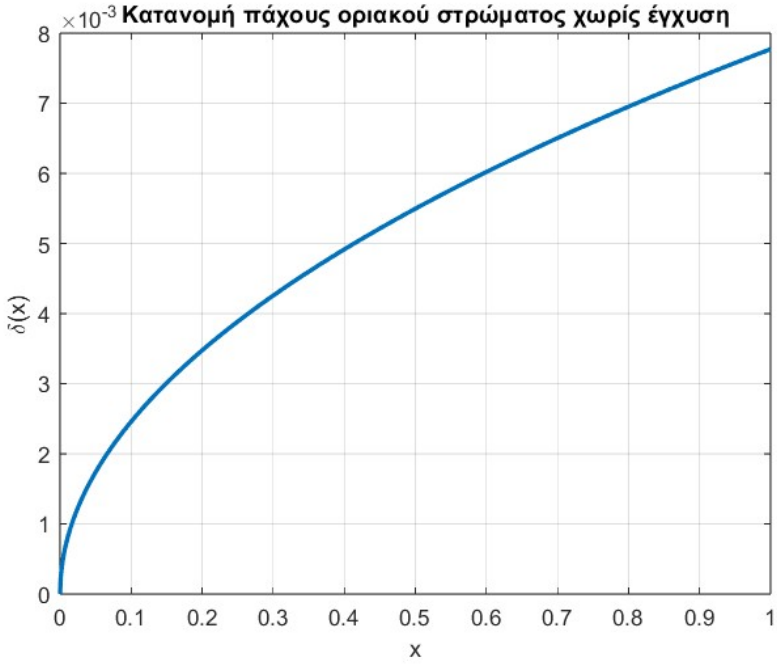
$$\frac{u}{U} = \sin\left(\frac{\pi}{2}\,\eta\right), \qquad \eta = \frac{y}{\delta(x)}$$

Συγκεκριμένα, αυτό υπολογίστηκε στη θέση x=L/2 και το προφίλ που προκύπτει φαίνεται στο σχήμα:

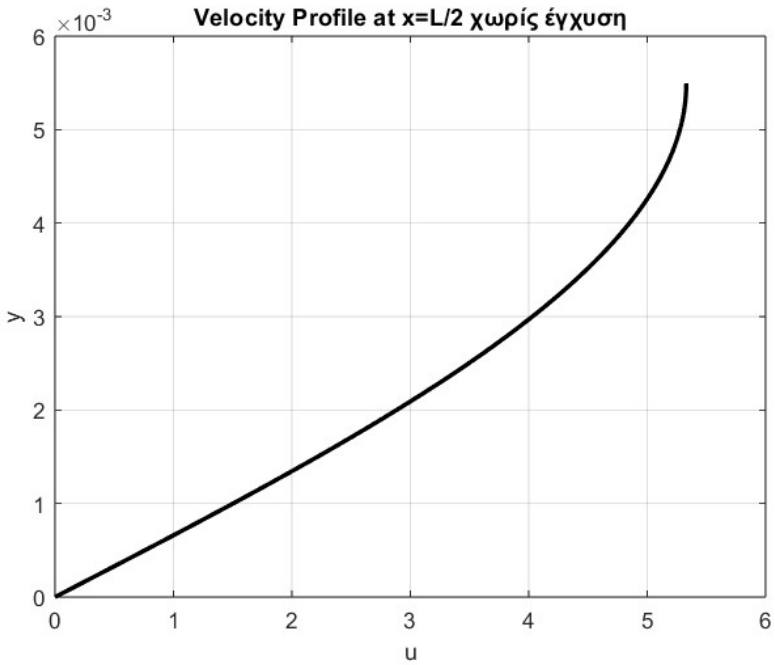


Σχήμα 3: Profile ταχύτητας για x=L/2

Τέλος, προκειμένου να φανεί η συνεισφορά του μηχανισμού έγχυσης, υπολογίζονται και σχεδιάζονται τα αντίστοιχα σχήματα πάχους οριακού στρώματος και προφίλ ταχύτητας για x=L/2 για την περίπτωση χωρίς τον μηχανισμό στην πλάκα:



Σχήμα 4: Πάχος οριακού στρώματος χωρίς έγχυση



Σχήμα 5: Profile ταχύτητας για $x = L/2$ χωρίς έγχυση

Παρατηρούμε πως χωρίς έγχυση η τάξη μεγέθους τόσο του πάχους οριακού στρώματος όσο και των ταχυτήτων, μειώνεται, γεγονός που αυξάνει τις τριβές στην επιφάνεια της πλάκας. Κρίνεται λοιπόν απαραίτητη η χρήση του μηχανισμού.

Ερώτημα 2: Finite Differences (FD)

Έχοντας το λογισμικό υπολογισμού της συνάρτησης F (το οποίο παρατίθεται και στο παράρτημα της εργασίας) με εισόδους τα a,b,c, αρκούν 6 κλήσεις του λογισμικού για τον υπολογισμό των παραγώγων της F. Συγκεκριμένα, οι ακριβείς σχέσεις των παραγώγων είναι:

$$\frac{\delta F}{\delta a} = \frac{F(a+h,b,c) - F(a-h,b,c)}{2h}$$

$$\frac{\delta F}{\delta b} = \frac{F(a,b+h,c) - F(a,b-h,c)}{2h}$$

$$\frac{\delta F}{\delta c} = \frac{F(a,b,c+h) - F(a,b,c-h)}{2h}$$

Στη μέθοδο των πεπερασμένων διαφορών, κρίνεται σημαντική η επιλογή του βήματος h για την ακρίβεια των υπολογισμών, καθώς πολύ μεγάλο βήμα μπορεί να δώσει πολύ κακή εκτίμηση της παραγώγου ενώ πολύ μικρό βήμα μπορεί ακόμα και να οδηγήσει σε απροσδιοριστία. Μετά από μελέτη συμπεριφοράς των παραγώγων ως προς το h, αποφάσισα πως το εύρος $10^{-6} < h < 10^{-3}$ είναι ασφαλές για τον υπολογισμό αυτών. Τα αριθμητικά αποτελέσματα της μεθόδου παρατίθονται συγκεντωτικά και με τις άλλες μεθόδους στο 7ο ερώτημα της εργασίας (βλ. σελ. 12).

Ερώτημα 3: Discrete Adjoint (DA)

Για την διακριτή συζηγή μέθοδο χρειάζεται να λύσουμε την συζηγή εξίσωση αρχικά για να βρούμε το συζηγές πεδίο Ψ:

$$\frac{\partial R}{\partial \delta}\boldsymbol{\Psi} = \frac{\partial F}{\partial \delta}$$

Προκειμένου να υπολογιστεί ο πίνακας $\frac{\partial R}{\partial \delta}$ χρειαζόμαστε την εξίσωση R στη διακριτή μορφή με την οποία την επιλύσαμε αρχικά, δηλαδή σε διακριτή μορφή Runge-Kutta 2ης τάξης. Η διακριτή μορφή της εξίσωσης και οι υπολογισμοί των τύπων για κάθε στοιχείο του πίνακα φαίνεται στις παρακάτω χειρόγραφες σημειώσεις:

Primal: $R = A\delta \frac{d\delta}{dx} + V_0(x)\delta - B = 0$, όπου $A = \left(\frac{4}{n} - \frac{1}{2}\right)V$ και $B = \frac{\pi}{2} \cdot V$

(Runge-Kutta 2ης τάξης)

$$\frac{d\delta}{dx} = \frac{B}{A} \cdot \frac{1}{\delta} - \frac{V_0(x)}{A} = f(x, \delta)$$

- $K_1 = f(x_i, \delta_i)$
- $\delta_{i+1/2} = \delta_i + \frac{\Delta x}{2} \cdot K_1$
- $K_2 = f\left(x_i + \frac{\Delta x}{2}, \delta_{i+1/2}\right)$
- $\delta_{i+1} = \delta_i + \Delta x \cdot K_2 = \delta_i + \Delta x\left(\frac{B}{A} \cdot \frac{1}{\delta_{i+1/2}} - \frac{V_0(x_i + \Delta x/2)}{A}\right) \Rightarrow$

$$\Rightarrow \delta_{i+1} = \delta_i + \Delta x \cdot \left(\frac{B}{A} \cdot \frac{1}{\delta_i + \frac{\Delta x}{2} \cdot K_1} - \frac{V_0(x_i + \Delta x/2)}{A}\right) \Rightarrow$$

$$\Rightarrow \delta_{i+1} = \delta_i + \Delta x \left(\frac{B}{A} \cdot \frac{1}{\delta_i + \frac{\Delta x}{2} \cdot \left(\frac{B}{A} \cdot \frac{1}{\delta_i} - \frac{V_0(x_i)}{A}\right)}\right) - \Delta x \cdot \frac{V_0(x_i + \Delta x/2)}{A}$$

$$\Rightarrow \delta_{i+1} = \delta_i + \Delta x \frac{B}{A\delta_i + \frac{B\Delta x}{2} \cdot \frac{1}{\delta_i} - \Delta x \cdot \frac{V_0(x_i)}{2}} - \Delta x \cdot \frac{V_0(x_i + \Delta x/2)}{A}$$

$$\Rightarrow \delta_{i+1} = \delta_i + \Delta x \cdot \frac{2B\delta_i}{2A\delta_i^2 - V_0(x_i)\Delta x \delta_i + B\Delta x} - \Delta x \frac{V_0(x_i + \Delta x/2)}{A}$$

Άρα τελικά:

$$R = \delta_{i+1} - \delta_i - \frac{2B\Delta x \delta_i}{2A\delta_i^2 - V_0(x_i)\Delta x \delta_i + B\Delta x} + \Delta x \frac{V_0(x_i + \Delta x/2)}{A}$$

Και άρα:

- $\frac{\partial R_i}{\partial \delta_{i+1}} = 1$
- $\frac{\partial R_i}{\partial \delta_i} = -1 - 2B\Delta x \cdot \frac{2A\delta_i^2 - V_0\Delta x \delta_i + B\Delta x - \delta_i(4A\delta_i - V_0\Delta x)}{(2A\delta_i^2 - V_0(x_i)\Delta x \delta_i + B\Delta x)^2} =$

$$= -1 - 2B\Delta x \cdot \frac{-2A\delta_i^2 + B\Delta x}{(2A\delta_i^2 - V_0\Delta x \delta_i + B\Delta x)^2}$$

Στη συνέχεια, μένει ο υπολογισμός του διανύσματος $\frac{\partial F}{\partial \delta}$. Γράφοντας την F σε διακριτοποιημένη μορφή με τη μέθοδο του τραπεζίου μπορούμε να υπολογίσουμε τις μερικές παραγώγους της ως προς κάθε δ_i. Συγκεκριμένα, προκύπτει ο τύπος για τα εσωτερικά στοιχεία του διανύσματος :

$$\frac{\partial F}{\partial \delta_i} = -c\,\Delta x\,\frac{1}{\delta_i^2}$$

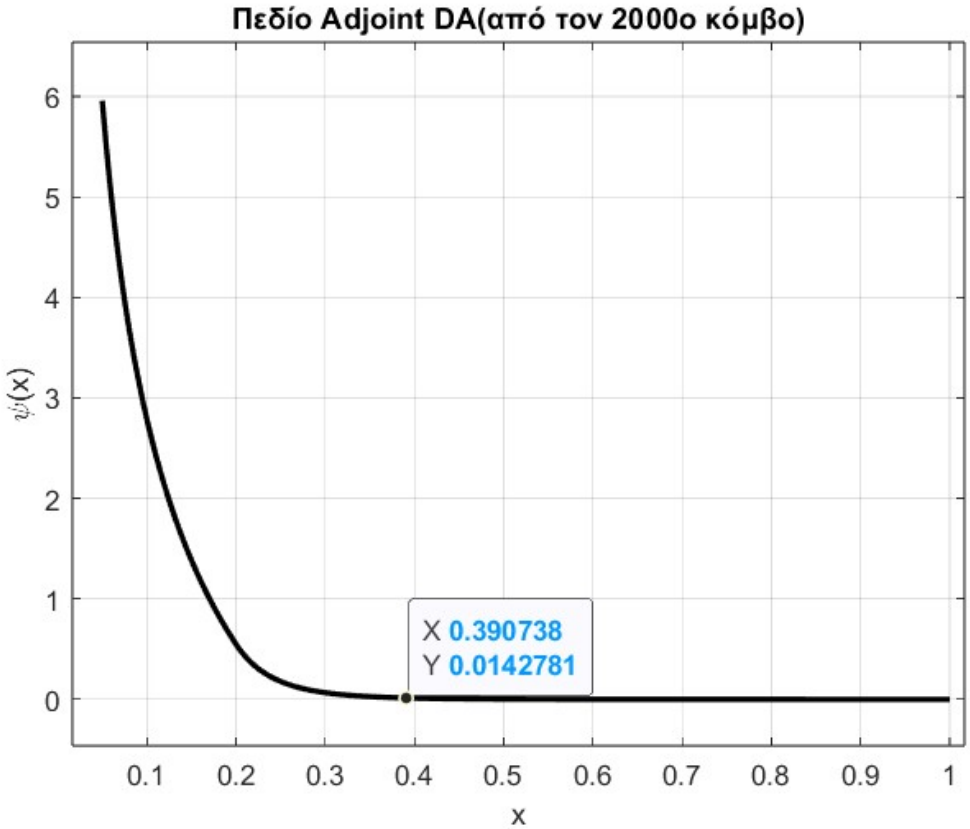
ενώ για τα άκρα:

$$\frac{\partial F}{\partial \delta_0} = -c\,\frac{\Delta x}{2}\,\frac{1}{\delta_0^2}$$

$$\frac{\partial F}{\partial \delta_N} = -c\,\frac{\Delta x}{2}\,\frac{1}{\delta_N^2}$$

όπου c ο συντελεστής $c = \frac{\pi}{2}\,\mu U$.

Επιλύοντας λοιπόν, αυτό το διδιαγώνιο σύστημα που προκύπτει υπολογίζουμε το συζηγές πεδίο της DA. Αυτό φαίνεται σχεδιασμένο στο κάτω σχήμα:



Σχήμα 6: Συζηγές Πεδίο DA

Στη συνέχεια υπολογίζουμε τα $\frac{\partial R}{\partial b_n}$ μέσω του τύπου:

$$\frac{\partial R}{\partial b_n} = -\frac{2B\,\Delta x^2\,\delta_i^2}{\left(2A\,\delta_i^2 + B\,\Delta x - v_0(x_i)\,\Delta x\right)^2}\,\frac{\partial v_0(x_i)}{\partial b_n} \;-\; \frac{\Delta x}{A}\,\frac{\partial v_0\left(x_i + \frac{\Delta x}{2}\right)}{\partial b_n}$$

και τελικά μέσα από τον παρακάτω τύπο υπολογίζουμε τις παραγώγους της F:

$$\frac{\delta F}{\delta \boldsymbol{b}} = \frac{\partial F}{\partial \boldsymbol{b}} - \boldsymbol{\Psi}^T\,\frac{\partial R}{\partial \boldsymbol{b}}$$

Ερώτημα 4: Direct Differentiation (DD)

Για την ευθεία διαφόριση της F, προκύπτει ο γενικός τύπος:

$$\frac{\delta F}{\delta b_n} = \frac{\partial F}{\partial b_n} + \frac{\partial F}{\partial u}\,\frac{\delta u}{\delta b_n}$$

όπου u=δ(x).

Αρχικά, επειδή η F δεν εξαρτάται άμεσα από τις μεταβλητές σχεδιασμού, η μερική παράγωγος ως προς αυτές είναι ίση με 0. Άρα τελικά:

$$\frac{\delta F}{\delta b_n} = \frac{\partial F}{\partial u}\,\frac{\delta u}{\delta b_n}$$

Απαιτείται λοιπόν, ο υπολογισμός των ολικών παραγώγων του οριακού στρώματος ως προς τις μεταβλητές σχεδιασμού. Για τον υπολογισμό αυτόν, χρειαζόμαστε την Primal σχέση:

$$R = \left(\frac{2}{\pi} - \frac{1}{2}\right)\frac{U}{\nu}\,\delta\,\frac{d\delta}{dx} + \frac{v_0(x)}{\nu}\,\delta - \frac{\pi}{2} = 0$$

Παραγωγίζοντας αυτή την σχέση ως προς b_n παίρνουμε:

$$\left(\frac{2}{\pi} - \frac{1}{2}\right)\frac{U}{\nu}\left(\frac{\delta\delta}{\delta b_n}\frac{d\delta}{dx} + \delta\,\frac{d}{dx}\left(\frac{\delta\delta}{\delta b_n}\right)\right) + \frac{1}{\nu}\left(\frac{\partial v_0}{\partial b_n}\,\delta + v_0(x)\,\frac{\delta\delta}{\delta b_n}\right) = 0$$

Θέτοντας $\dfrac{\delta\delta}{\delta b_n} = \phi_i$ προκύπτει:

$$\left(\frac{2}{\pi} - \frac{1}{2}\right)\frac{U}{\nu}\,\delta\,\frac{d\phi_i}{dx} + \left[\left(\frac{2}{\pi} - \frac{1}{2}\right)\frac{U}{\nu}\,\frac{d\delta}{dx} + \frac{v_0(x)}{\nu}\right]\phi_i + \frac{\delta}{\nu}\,\frac{\partial v_0}{\partial b_n} = 0$$

Δηλαδή, χρειάζεται να λύσουμε 3 διαφορικές εξισώσεις με άγνωστες τις παραγώγους του δ ως προς κάθε μεταβλητή σχεδιασμού. Η επίλυση αυτών, γίνεται και εδώ με RK2 με αρχικές συνθήκες $\phi_i(0) = 0$.

Όσο για το διάνυσμα $\dfrac{\partial F}{\partial \delta}$, είναι ήδη υπολογισμένο από το προηγούμενο ερώτημα (DA).

Έτσι λοιπόν, πολλαπλασιάζοντας τα δύο διανύσματα, παίρνουμε τις παραγώγους της F για κάθε μεταβλητή σχεδιασμού:

$$\frac{\delta F}{\delta b_n} = \frac{\partial F}{\partial \delta}\,\phi_i$$

Ερώτημα 5: Continuous Adjoint (CA)

Για την εξαγωγή της συζυγούς εξίσωσης με τη μέθοδο CA χρειάζεται να γραφεί η F στην επαυξημένη της μορφή. Δηλαδή:

$$F_{\text{aug}} = F + \int_0^L \psi\, R\, dx$$

Παραγωγίζοντας την παραπάνω σχέση προκύπτει η FAE εξίσωση με τη συνοριακή συνθήκη (ABC) όπως φαίνεται στις παρακάτω χειρόγραφες σημειώσεις:

Continuous Adjoint

$$F_{aug} = F + \int_0^L \psi R\, dx = \int_0^L c \cdot \frac{1}{\delta}\, dx + \int_0^L \psi \left(A\delta \frac{d\delta}{dx} + v_0(x)\delta - B\right) dx$$

$$\Rightarrow \frac{\delta F_{aug}}{\delta b_n} = \int_0^L c \cdot \left(-\frac{1}{\delta^2}\right) \frac{\delta\delta}{\delta b_n}\, dx + \int_0^L \cancel{\psi A \frac{d\delta}{dx} \cdot \frac{\delta\delta}{\delta b_n}}\, dx + \underbrace{\int_0^L \psi A \delta \frac{d}{dx}\left(\frac{\delta\delta}{\delta b_n}\right) dx}_{T1}$$

$$+ \int_0^L \psi \cdot v_0 \cdot \frac{\delta\delta}{\delta b_n}\, dx + \int_0^L \psi \delta \frac{\delta v_0}{\delta b_n}\, dx$$

$$T1: \left.\psi A \delta \frac{\delta\delta}{\delta b_n}\right|_0^L - \int_0^L A\delta \frac{d\psi}{dx} \cdot \frac{\delta\delta}{\delta b_n}\, dx - \int_0^L \cancel{A\psi \frac{d\delta}{dx} \cdot \frac{\delta\delta}{\delta b_n}}$$

FAE

$$-A\delta \frac{d\psi}{dx} + v_0 \cdot \psi - \frac{c}{\delta^2} = 0$$

ABC

$$\left.\psi A \delta \frac{\delta\delta}{\delta b_n}\right|_0^L = \left.\psi A \delta \frac{\delta\delta}{\delta b_n}\right|_L - \cancel{\left.\psi A \delta \frac{\delta\delta}{\delta b_n}\right|_0} \Rightarrow \boxed{\psi|_L = 0} \rightarrow \text{ABC}$$

0 γιατί $\delta(0) = \varepsilon$

Όπως φαίνεται η Δ.Ε. που προκύπτει για το συζηγές πεδίο είναι γραμμική ως προς το άγνωστο Ψ, σε αντίθεση με την εξίσωση του primal που είναι μη γραμμική ως προς δ. Επίσης, η συνοριακή συνθήκη του adjoint είναι στο x=L ενώ για το Primal πρόβλημα στο x=0. Αυτό σημαίνει πως η φορά διάδοσης της πληροφορίας στην adjoint εξίσωση θα πηγαίνει ανάποδα από το Primal (από τα δεξιά προς τα αριστερά) καθώς την επιλύουμε με Runge-Kutta.

Τέλος, από τον όρο που μένει προκύπτουν και οι τύποι για τις παραγώγους (Sensitivity Derivatives, SD) όπως φαίνεται παρακάτω:

Με τον όρο που απομένει βγάζουμε τον τύπο για τις παραγώγους ευαισθησία

$$SD: \frac{\delta F}{\delta b_n} = \int_0^L \psi\, \delta \cdot \frac{\delta v_0}{\delta b_n}\, dx$$

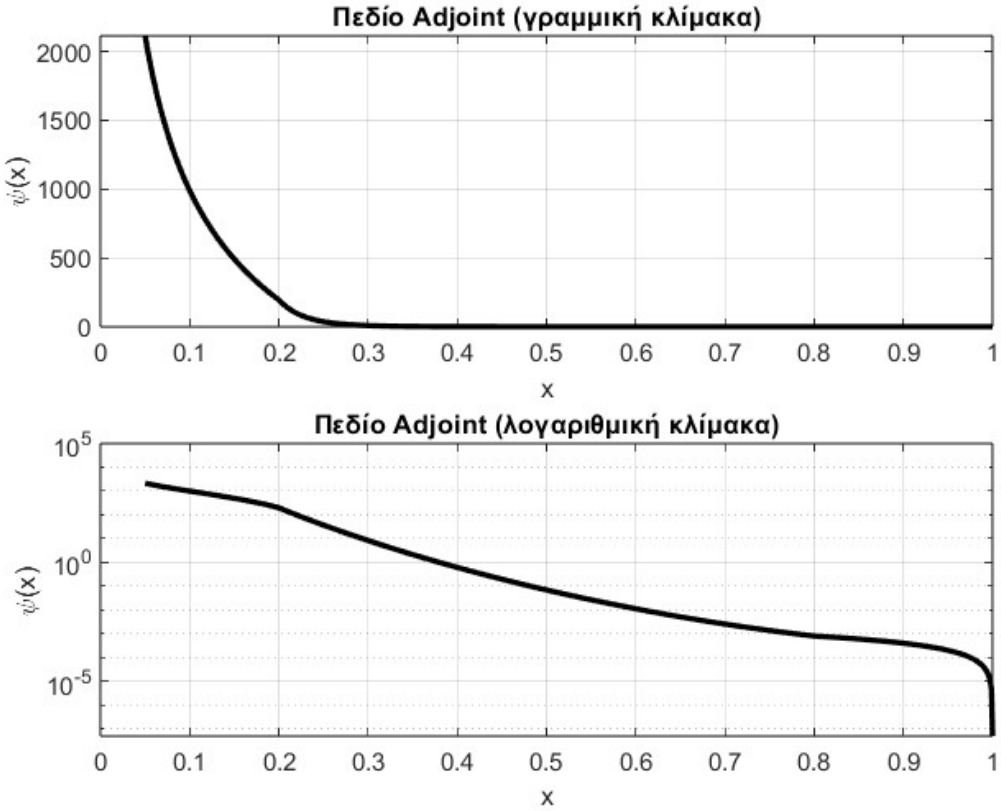
$$\frac{\delta F}{\delta a} = \int_0^L \psi\, \delta\, (-2)\left[a x^2 + b x + c \cdot \sin\left(\frac{\pi x}{6}\right)\right] x^2\, dx$$

$$\frac{\delta F}{\delta b} = \int_0^L \psi\, \delta\, (-2)\left[a x^2 + b x + c \sin\left(\frac{\pi x}{6}\right)\right] x\, dx$$

$$\frac{\delta F}{\delta c} = \int_0^L \psi\, \delta\, (-2)\left[a x^2 + b x + c \sin\left(\frac{\pi x}{6}\right)\right] \sin\left(\frac{\pi x}{6}\right)$$

Ερώτημα 6: Λογισμικό επίλυσης του συνεχούς συζυγούς προβλήματος (CA) και εύρεσης των παραγώγων ευαισθησίας

Στο σημείο αυτό αναπτύχθηκε λογισμικό επίλυσης της FAE με Runge-Kutta 2ης τάξης με φορά από x=L —> x=0 όπως αναφέρθηκε και παραπάνω. Από την επίλυση προκύπτει το παρακάτω συζηγές πεδίο σχεδιασμένο σε γραμμική και λογαριθμική κλίμακα:



Σχήμα 7: Συζηγές Πεδίο CA

Το adjoint πεδίο είναι μεγάλο κοντά στην είσοδο και σχεδόν μηδενίζεται προς την έξοδο. Αυτό σημαίνει ότι οι αλλαγές στην αρχή του πεδίου επηρεάζουν πολύ περισσότερο το αποτέλεσμα από τις αλλαγές στο τέλος.

Τέλος, υπολογίζουμε τις παραγώγους ευαισθησίας σύμφωνα με τους τύπους που ααφέρθηκαν παραπάνω, υπολογίζοντας τα ολοκληρώματα που χρειάζεται με την μέθοδο του τραπεζίου.

Ερώτημα 7: Σύγκριση Μεθόδων

Παρακάτω φαίνονται τα συγκεντρωτικά αποτελέσματα για τους υπολογισμούς των παραγώγων για κάθε μέθοδο για το σετ των a,b,c που επέλεξα (σελ. 1):

	FD	DA	DD	CA
$\delta F/\delta a$	$-1.3562657264630 \times 10^{-3}$	$-1.3923704551240 \times 10^{-3}$	$-1.3562657106070 \times 10^{-3}$	$-1.3562063794600 \times 10^{-3}$
$\delta F/\delta b$	$-4.4103335792440 \times 10^{-3}$	$-4.5654887537080 \times 10^{-3}$	$-4.4103336278330 \times 10^{-3}$	$-4.4104720828860 \times 10^{-3}$
$\delta F/\delta c$	$-2.2979590938440 \times 10^{-3}$	$-2.3789926751800 \times 10^{-3}$	$-2.2979591745900 \times 10^{-3}$	$-2.2978095280610 \times 10^{-3}$

Πίνακας 1: Παράγωγοι της F ως προς τις μεταβλητές σχεδιασμού

Παρατηρείται ότι όλες οι μέθοδοι δίνουν πολύ κοντινές τιμές για τις παραγώγους του F. Οι μικρές διαφορές που εμφανίζονται οφείλονται σε διαφορετικές

αριθμητικές προσεγγίσεις και στη διακριτοποίηση του προβλήματος και θεωρούνται αμελητέες. Συνεπώς, τα αποτελέσματα των μεθόδων κρίνονται συνεπή.

Επιπλέον, παρατίθεται ο πίνακας με το κόστος κάθε μεθόδου ανά κύκλο βελτιστοποίησης:

Μέθοδος	**Υπολογιστικό κόστος (TU)**
Finite Differences (FD)	$2N$
Direct Differentiation (DD)	$N+1$
Discrete Adjoint (DA)	2
Continuous Adjoint (CA)	2

Πίνακας 2: Συγκριτικό υπολογιστικό κόστος των μεθόδων υπολογισμού παραγώγων

Ερώτημα 8: Διαδικασία Βελτιστοποίησης

Αφού αναπτύχθηκε το λογισμικό για την εύρεση των παραγώγων μέσω του συνεχούς adjoint, ξεκινάμε την διαδικασία ελαχιστοποίησης της F. Για την διαδικασία βελτιστοποίησης θα ακολουθήσουμε την μέθοδο ALM (Augmented Lagrange Multiplier) ώστε να λάβουμε υπόψην μας και τον περιορισμό ισότητας του προβλήματος. Αυτός, λύνοντας το ολοκλήρωμα της παροχής, έχει τη μορφή που φαίνεται παρακάτω:

$$\begin{aligned} C1 = 0.0444450027370472\,c^2 + (0.104737806763403\,a + 0.172815991859899\,b)\,c \\ + \left(0.065472\,a^2 + 0.204\,ab + 0.168\,b^2 - 0.05\,U\right) = 0 \end{aligned}$$

Έχοντας την σχέση του περιορισμού μπορούμε να γράψουμε την επαυξημένη Lagrange του προβλήματος μας:

$$\Phi = F - \lambda_1\,C_1 + \omega_P\,C_1^2$$

όπου λ_1 ο συντελεστής Lagrange, ενώ ω_P ο διορθωτικός πολλαπλασιαστής που ουσιαστικά 'σπρώχνει' την διαδικασία προς την καμπύλη του περιορισμού.

Παραγωγίζοντας την παραπάνω σχέση βρίσκουμε την παράγωγο της επαυξημένης, η οποία μας δείχνει και την κατεύθυνση που πρέπει να κινηθούμε για να βρούμε τη βέλτιστη λύση:

$$\frac{\delta\Phi}{\delta b_n} = \frac{\delta F}{\delta b_n} - \lambda_1\frac{\delta C_1}{\delta b_n} + 2\,\omega_P\,C_1\frac{\delta C_1}{\delta b_n}$$

Τον όρο $\frac{\delta C_1}{\delta b_n}$ τον βρίσκουμε εύκολα παραγωγίζοντας την σχέση περιορισμού ως προς κάθε μεταβλητή, ενώ για τον όρο $\frac{\delta F}{\delta b_n}$ καλούμε το λογισμικό του CA που υλοποιήσαμε στο 7ο ερώτημα.

Στόχος είναι η ελαχιστοποίηση της δύναμης αντίστασης στην πλάκα, η οποία είναι αντιστρόφως ανάλογη του πάχους οριακού στρώματος δ. Το δ από την άλλη

μπορούμε εύκολα να δούμε ότι μεγαλώνει όσο μεγαλώνει και το v_0. Για τον λόγο αυτό επέλεξα αρχικοποίηση για την βελτιστοποίηση, ένα σετ των a,b,c ώστε να τηρείται οριακά ο περιορισμός και το a που είναι ο πολλαπλασιαστής του τετραγωνικού όρου να είναι σχετικά μεγάλο. Όσον αφορά τον συντελεστή Lagrange γνωρίζουμε από την 4η συνθήκη KKT πως προκειμένου να βρεθεί η βέλτιστη λύση πρέπει αυτός ο συντελεστής να έχει αρνητική τιμή. Συνεπώς, του έδωσα μια αρχικοποίηση που να σέβεται αυτή τη συνθήκη. Παρακάτω φαίνονται οι τιμές αρχικοποίησης:

Μεταβλητή	**Αρχική τιμή**
a	11
b	-53
c	89
λ_1	-0.01

Πίνακας 3: Αρχικοποίηση μεταβλητών για τη μέθοδο Augmented Lagrangian

Όσον αφορά την ανανέωση των μεταβλητών σχεδιασμού σε κάθε κύκλο βελτιστοποίησης, χρησιμοποίησα τη μέθοδο απότομης καθόδου (steepest descent) με τον τύπο που φαίνεται παρακάτω:

$$b_n^{(k+1)} \;=\; b_n^{(k)} \;-\; \eta\,\frac{\partial \Phi}{\partial b_n}, \qquad n = 1, 2, 3$$

ενώ για την ανανέωση του λ_1 :

$$\lambda_1^{(k+1)} \;=\; \lambda_1^{(k)} \;-\; 2\omega_P\, C_1$$

Περιγραφή ALM

- **Inner loop (unconstrained minimization)**: Για δεδομένα (λ_1, ω) ελαχιστοποιούμε ως προς b το $\Phi(b; \lambda_1, \omega)$ με τη μέθοδο steepest descent.
- **Outer loop (update multipliers)**: Μετά την ελαχιστοποίηση, ανανεώνουμε τον πολλαπλασιαστή λ_1 με βάση το residual του περιορισμού, ώστε η λύση να οδηγείται σταδιακά στην εφικτή περιοχή $c(b) = 0$.

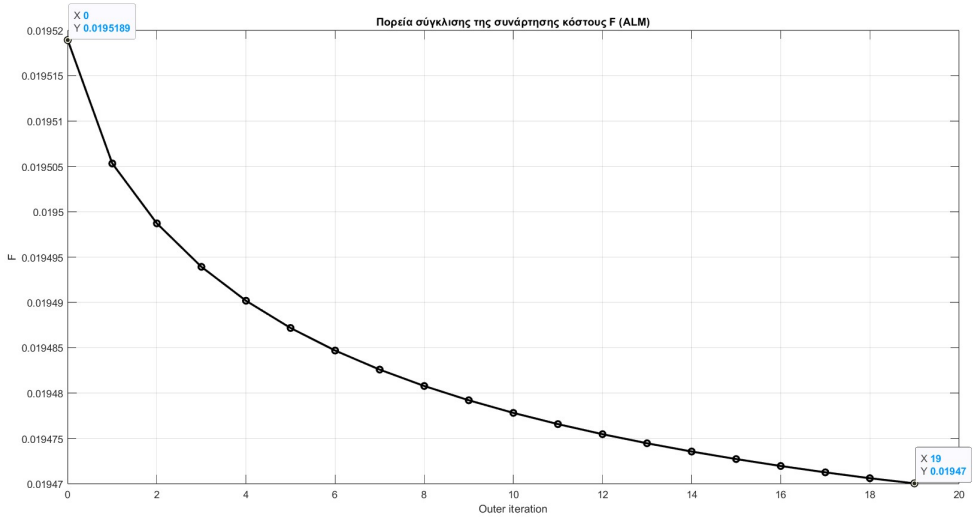
Ξεκινώντας τη διαδικασία επέλαξα $\omega_P = 5$ ενώ σε κάθε ανανέωση του εξωτερικού loop αύξανα το ω_P με έναν πολλαπλασιαστή $\gamma = 1.1$. (Επειδή ξεκίνησα από σετ τιμών που τηρούσαν τον περιορισμό δε χρειαζόταν τόσο μεγάλη τιμή για το penalty για αυτό διάλεξα να κρατήσω αυτή την παράμετρο σε χαμηλή τιμή)

Όσον αφορά το η για το inner loop, επέλεξα σταθερό $\eta = 3$ καθώς αυτό ήταν το μέγιστο βήμα που έδινε μονοτονική πορεία για τους 20 κύκλους βελτιστοποίησης που εκτέλεσα.

Παρακάτω φαίνεται ο πίνακας των τελικών αποτελεσμάτων της διαδικασίας βελτιστοποίησης καθώς και η πορεία της F κατά τη διάρκεια των 20 κύκλων:

Μεταβλητή	Τελική τιμή
F	$1.947004100312 \times 10^{-2}$
a	10.67026973631
b	-53.04714463766
c	89.84490506974
λ_1	$-3.487071645762 \times 10^{-3}$
c_{con}	$-9.607494973807 \times 10^{-6}$

Πίνακας 4: Τελική λύση της μεθόδου Augmented Lagrangian



Σχήμα 8: Πορεία Βελτιστοποίησης

Παρότι η συνολική μείωση της αντικειμενικής συνάρτησης είναι περιορισμένη (≈0.25%), αυτό είναι αναμενόμενο, καθώς η αρχική λύση ήταν ήδη κοντά στο βέλτιστο και ο περιορισμός περιορίζει σημαντικά τον εφικτό χώρο σχεδιασμού.

Συμπέρασμα

Συνολικά, η εφαρμογή της μεθόδου Augmented Lagrangian επέτρεψε την αποτελεσματική ενσωμάτωση του περιορισμού παροχής στη διαδικασία βελτιστοποίησης, χωρίς να διαταραχθεί η σταθερότητα της σύγκλισης. Η χρήση του συνεχούς συζυγούς πεδίου (Continuous Adjoint) κατέστησε δυνατή την αποδοτική και ακριβή εκτίμηση των παραγώγων ευαισθησίας, ανεξάρτητα από το πλήθος των μεταβλητών σχεδιασμού. Τα τελικά αποτελέσματα δείχνουν ότι ο περιορισμός ικανοποιείται εντός της επιθυμητής ανοχής, ενώ η αντικειμενική συνάρτηση παρουσιάζει ομαλή και μονοτονική μείωση. Συνεπώς, η προτεινόμενη μεθοδολογία κρίνεται κατάλληλη για την επίλυση παρόμοιων προβλημάτων βελτιστοποίησης ροών με αντίστοιχους περιορισμούς.

ΚΩΔΙΚΑΣ ΓΙΑ ΤΗΝ ΕΡΓΑΣΙΑ ΒΕΛΤΙΣΤΟΠΟΙΗΣΗΣ ADJOINT

```
#include <cmath>

#include <iostream>

#include <vector>

#include <cmath>

#include <fstream>

#include <iomanip>

#include <string>

#include <algorithm>


static const double U   = 16.0/3.0;

double Q = -0.05 * U;

static const double mi  = 1.75e-5;

static const double rho = 1.25;

static const double nu  = mi / rho;

static const double eps = 1e-6;

static const double A0 = M_PI*nu/2;

static const double C = M_PI*mi*U/2;


static inline double v0(double x, double a, double b, double c)

{

   if (x <= 0.2 || x >= 0.8) return 0.0;

   double s = std::sin(M_PI * x / 6.0);

   double g = a*x*x + b*x + c*s;

   return -(g*g);

}


static inline double ddelta_dx(double x, double delta,

                               double a, double b, double c)
```

```
{
    const double K = (2.0 / M_PI - 0.5);

    delta = std::max(delta, 1e-14);
    double v = v0(x, a, b, c);

    return ( (0.5 * M_PI) * nu - v * delta ) / (K * U * delta);
}

double computeF(double a, double b, double c,
                int N,
                bool writeFiles = true,
                const std::string& deltaFile = "delta.txt",
                const std::string& fricFile  = "friction.txt")
{
    const double x0 = 0.0, x1 = 1.0;
    const double dx = (x1 - x0) / (N - 1);

    std::vector<double> x(N), delta(N), tauw(N);
    for (int i = 0; i < N; ++i) x[i] = x0 + i * dx;

    delta[0] = eps;

    for (int i = 0; i < N - 1; ++i)
    {
        double xi = x[i];
        double k1 = ddelta_dx(xi,          delta[i],              a, b, c);
        double k2 = ddelta_dx(xi + dx/2.0, delta[i] + dx*k1/2.0,  a, b, c);

        delta[i+1] = delta[i] + dx * k2;
        if (delta[i+1] < eps) delta[i+1] = eps;
```

```
    }

    for (int i = 0; i < N; ++i) {
        delta[i] = std::max(delta[i], 1e-14);
        tauw[i]  = (M_PI/2.0) * mi * U / delta[i];
    }

    double F = 0.0;
    for (int i = 0; i < N - 1; ++i) {
        F += 0.5 * (tauw[i] + tauw[i+1]) * dx;
    }

    if (writeFiles) {
        std::ofstream fdelta(deltaFile);
        fdelta << std::scientific << std::setprecision(17);
        for (int i = 0; i < N; ++i) fdelta << x[i] << " " << delta[i] << "\n";

        std::ofstream ffric(fricFile);
        ffric << std::scientific << std::setprecision(17);
        for (int i = 0; i < N; ++i) ffric << x[i] << " " << tauw[i] << "\n";
    }

    return F;
}

static inline double dv0_da(double x, double a, double b, double c)
{
    if (x <= 0.2 || x >= 0.8) return 0.0;

    double s = std::sin(M_PI * x / 6.0);
    double g = a*x*x + b*x + c*s;
```

```
    return -2.0 * g * (x*x);
}

static inline double dv0_db(double x, double a, double b, double c)
{
    if (x <= 0.2 || x >= 0.8) return 0.0;

    double s = std::sin(M_PI * x / 6.0);
    double g = a*x*x + b*x + c*s;

    return -2.0 * g * x;
}

static inline double dv0_dc(double x, double a, double b, double c)
{
    if (x <= 0.2 || x >= 0.8) return 0.0;

    double s = std::sin(M_PI * x / 6.0);
    double g = a*x*x + b*x + c*s;

    return -2.0 * g * s;
}


static inline double rhs(double delta_theta, double delta, double v0_theta,
                         double A0, double K, double U)
{
    const double KU = K * U;
    const double eps = 1e-14;
    if (std::fabs(delta) < eps) delta = (delta >= 0.0 ? eps : -eps);

    return -(A0 / KU) * (delta_theta / (delta * delta)) - (1.0 / KU) * v0_theta;
```

```
}

void solve_sens_RK2(
    int N, double x0, double h,
    const double* x,
    const double* delta,
    double a, double b, double c,
    double A0, double K, double U,
    double da0, double db0, double dc0,
    double* da, double* db, double* dc
){
    da[0] = da0;
    db[0] = db0;
    dc[0] = dc0;

    for (int i = 0; i < N-1; ++i)
    {
        double xi   = x ? x[i] : (x0 + i*h);
        double xmid = xi + 0.5*h;

        double di   = delta[i];
        double dmid = 0.5 * (delta[i] + delta[i+1]);

        // --- a ---
        double k1a = rhs(da[i], di,   dv0_da(xi,   a,b,c), A0,K,U);
        double k2a = rhs(da[i] + 0.5*h*k1a, dmid, dv0_da(xmid, a,b,c), A0,K,U);
        da[i+1] = da[i] + h*k2a;

        // --- b ---
        double k1b = rhs(db[i], di,   dv0_db(xi,   a,b,c), A0,K,U);
        double k2b = rhs(db[i] + 0.5*h*k1b, dmid, dv0_db(xmid, a,b,c), A0,K,U);
```

```
        db[i+1] = db[i] + h*k2b;


        // --- c ---
        double k1c = rhs(dc[i], di,   dv0_dc(xi,   a,b,c), A0,K,U);
        double k2c = rhs(dc[i] + 0.5*h*k1c, dmid, dv0_dc(xmid, a,b,c), A0,K,U);
        dc[i+1] = dc[i] + h*k2c;
    }
}
static void solve_delta_RK2(int N, double a, double b, double c,
                            std::vector<double>& x, std::vector<double>& delta)
{
    const double x0 = 0.0, x1 = 1.0;
    const double dx = (x1 - x0) / (N - 1);

    x.resize(N);
    delta.resize(N);

    for (int i = 0; i < N; ++i) x[i] = x0 + i * dx;
    delta[0] = eps;
    for (int i = 0; i < N - 1; ++i)
    {
        double xi = x[i];

        double k1 = ddelta_dx(xi,          delta[i],              a, b, c);
        double k2 = ddelta_dx(xi + dx/2.0, delta[i] + dx*k1/2.0,  a, b, c);

        delta[i+1] = delta[i] + dx * k2;
        if (delta[i+1] < eps) delta[i+1] = eps;
    }
}
```

```
void compute_df_ddelta(const std::vector<double>& delta,
                       double dx,
                       std::vector<double>& df_ddelta)
{
   int N = (int)delta.size();
   df_ddelta.resize(N);

   const double C = (M_PI/2.0) * mi * U;

   for (int i = 0; i < N; ++i)
   {
      double di = std::max(delta[i], 1e-14);

      double w = (i == 0 || i == N-1) ? 0.5 : 1.0;
      df_ddelta[i] = -C * dx * w / (di * di);
   }
}

double dot(const std::vector<double>& v1,
           const std::vector<double>& v2)
{
   double s = 0.0;
   int N = (int)v1.size();
   for (int i = 0; i < N; ++i)
      s += v1[i] * v2[i];
   return s;
}
// ADJOINT
static inline double adj_rhs(double x, double lam,
                             double delta, double A, double B, double C, double a, double b, double c)
{
```

```
    return ((v0(x,a,b,c))/(A*delta))*lam - (C/A)*(1.0/(delta*delta*delta));
}

void solve_adjoint_RK2_backward(
    int N,
    const double* x,
    const double* delta,
    double A, double B, double C,
    double a, double b, double c,
    double lamL,            // boundary condition at x=L
    double* lam
)
{

    lam[N-1] = lamL;

    for (int i = N - 1; i > 0; --i)
    {
        double xi   = x[i];
        double him1 = x[i-1];
        double h    = him1 - xi;

        double xmid = xi + 0.5 * h;

        double d_i   = delta[i];
        double d_mid = 0.5 * (delta[i] + delta[i-1]);

        double k1 = adj_rhs(xi,         lam[i],
                            d_i, A, B, C, a, b, c);
```

```
        double k2 = adj_rhs(xmid,        lam[i] + 0.5*h*k1,
                            d_mid, A, B, C, a, b, c);
        lam[i-1] = lam[i] + h * k2;
    }
}

double trapz_lam_delta_q(const std::vector<double>& x,
                         const std::vector<double>& lam,
                         const std::vector<double>& delta,
                         const std::vector<double>& q)
{
    int N = (int)x.size();
    double I = 0.0;

    for (int i = 0; i < N - 1; ++i)
    {
        double dx = x[i+1] - x[i];
        double di   = std::max(delta[i],   1e-14);
        double dip1 = std::max(delta[i+1], 1e-14);

        double gi   = lam[i]   * di   * q[i];
        double gip1 = lam[i+1] * dip1 * q[i+1];

        I += 0.5 * (gi + gip1) * dx;
    }
    return I;
}

// ALM
double compute_F_and_grad_CA(double a, double b, double c, int N,
                             double& dFda, double& dFdb, double& dFdc)
```

```
{

    std::vector<double> x, delta;
    solve_delta_RK2(N, a, b, c, x, delta);

    std::vector<double> lam(N);
    double lamL = 0.0;

    double A = (2/M_PI - 0.5)*U;
    double B = (M_PI*nu/2);

    solve_adjoint_RK2_backward(
        N, x.data(), delta.data(),
        A, B, C, a, b, c,
        lamL, lam.data()
    );

    std::vector<double> q_a(N), q_b(N), q_c(N);
    for (int i = 0; i < N; ++i) {
        q_a[i] = dv0_da(x[i], a, b, c);
        q_b[i] = dv0_db(x[i], a, b, c);
        q_c[i] = dv0_dc(x[i], a, b, c);
    }

    double Ia = trapz_lam_delta_q(x, lam, delta, q_a);
    double Ib = trapz_lam_delta_q(x, lam, delta, q_b);
    double Ic = trapz_lam_delta_q(x, lam, delta, q_c);

    dFda = Ia;
    dFdb = Ib;
    dFdc = Ic;
```

```
    return computeF(a, b, c, N, false);
}

double compute_c_and_grad(double a, double b, double c,
                          double& dcda, double& dcdb, double& dcdc)
{
    // Constraint:
    // 0.0444450027370472 c^2
    // + (0.104737806763403 a + 0.172815991859899 b) c
    // + (0.065472 a^2 + 0.204 a b + 0.168 b^2 - 0.05 U) = 0

    constexpr double K2   = 0.0444450027370472;
    constexpr double Ka_c = 0.104737806763403;
    constexpr double Kb_c = 0.172815991859899;

    constexpr double Kaa  = 0.065472;
    constexpr double Kab  = 0.204;
    constexpr double Kbb  = 0.168;

    // residual cval
    const double cval =
        K2 * c * c
        + (Ka_c * a + Kb_c * b) * c
        + (Kaa * a * a + Kab * a * b + Kbb * b * b - 0.05 * U);


    dcda = Ka_c * c + (2.0 * Kaa * a + Kab * b);
    dcdb = Kb_c * c + (Kab * a + 2.0 * Kbb * b);
    dcdc = 2.0 * K2 * c + (Ka_c * a + Kb_c * b);

    return cval;
```

```
}
int main()
{
    int N =80001;
    double h = 1e-6;

    double a = 11.0;
    double b = -53.0;
    double c = 89.86;

    double F0 = computeF(a,b,c,N,true);

    // ΠΕΠΕΡΑΣΜΕΝΕΣ ΔΙΑΦΟΡΕΣ
    double Fa_p = computeF(a+h,b,c,N,false);
    double Fa_m = computeF(a-h,b,c,N,false);
    double Fb_p = computeF(a,b+h,c,N,false);
    double Fb_m = computeF(a,b-h,c,N,false);
    double Fc_p = computeF(a,b,c+h,N,false);
    double Fc_m = computeF(a,b,c-h,N,false);

    double dF_da = (Fa_p - Fa_m) / (2*h);
    double dF_db = (Fb_p - Fb_m) / (2*h);
    double dF_dc = (Fc_p - Fc_m) / (2*h);

    std::cout << std::scientific << std::setprecision(12);
    std::cout << "F = " << F0 << "\n";
    std::cout << "dF/da (FD) = " << dF_da << "\n";
    std::cout << "dF/db (FD) = " << dF_db << "\n";
    std::cout << "dF/dc (FD) = " << dF_dc << "\n";
    std::cout << "N  = " << N << "\n";
    std::cout << "dx = " << 1.0/(N-1) << "\n";
```

```
// DD
std::vector<double> x, delta;
solve_delta_RK2(N, a, b, c, x, delta);

std::vector<double> da(N), db(N), dc(N);
double da0 = 0.0, db0 = 0.0, dc0 = 0.0;

const double dx = 1.0 / (N - 1);
const double Kc = (2.0 / M_PI - 0.5);

solve_sens_RK2(
    N, 0.0, dx,
    x.data(),
    delta.data(),
    a, b, c,
    A0, Kc, U,
    da0, db0, dc0,
    da.data(), db.data(), dc.data()
);

// π.χ. έλεγχος ότι όντως βγήκαν:
// std::cout << "delta_a(1) = " << da.back() << "\n";
// std::cout << "delta_b(1) = " << db.back() << "\n";
// std::cout << "delta_c(1) = " << dc.back() << "\n";

std::vector<double> df_ddelta;
compute_df_ddelta(delta, dx, df_ddelta);

double dF_da_DD = dot(df_ddelta, da);
double dF_db_DD = dot(df_ddelta, db);
double dF_dc_DD = dot(df_ddelta, dc);
```

```
std::cout << "---------------------------- " <<  "\n";
std::cout << "dF/da (DD) = " << dF_da_DD << "\n";
std::cout << "dF/db (DD) = " << dF_db_DD << "\n";
std::cout << "dF/dc (DD) = " << dF_dc_DD << "\n";

//ADJOINT
std::vector<double> lam(N);
double lamL = 0;
double A = (2/M_PI - 0.5)*U;
double B = (M_PI*nu/2);

solve_adjoint_RK2_backward(
    N,
    x.data(),
    delta.data(),
    A, B, C,
    a, b, c,
    lamL,
    lam.data()
);

std::vector<double> q_a(N), q_b(N), q_c(N);

for (int i = 0; i < N; ++i)
{
    q_a[i] = dv0_da(x[i], a, b, c);
    q_b[i] = dv0_db(x[i], a, b, c);
    q_c[i] = dv0_dc(x[i], a, b, c);
}

double Ia = trapz_lam_delta_q(x, lam, delta, q_a);
```

```
double Ib = trapz_lam_delta_q(x, lam, delta, q_b);
double Ic = trapz_lam_delta_q(x, lam, delta, q_c);

std::cout << "---------------------------- " <<  "\n";
std::cout << "Integral_a = " << Ia << "\n";
std::cout << "Integral_b = " << Ib << "\n";
std::cout << "Integral_c = " << Ic << "\n";
std::ofstream flam("adjoint_lambda.txt");
flam << std::scientific << std::setprecision(17);

for (int i = 0; i < N; ++i)
{
    flam << x[i] << " " << lam[i] << "\n";
}

flam.close();
// DISCRETE ADJOINT
std::vector<double> diag(N);
std::vector<double> psi(N);

diag[0]=1;

for (int i = 1; i < N; ++i)
{
    double di = delta[i];
    double xi = x[i];

    diag[i] = -1.0 - 2*B*dx *((-2)*A*di*di + B*dx)/((2*A*di*di - v0(xi,a,b,c)*dx*di + B*dx)*(2*A*di*di -
v0(xi,a,b,c)*dx*di + B*dx)) ;
}
```

```
psi[N-1] = df_ddelta[N-1];
for (int i = N-2; i >= 1; --i)
{
   psi[i] = (df_ddelta[i] - psi[i+1])/(diag[i]);
   }
   psi[0] = df_ddelta[0];

   std::ofstream psi_file("adjoint_psi.txt");

if (!psi_file.is_open())
{
   std::cerr << "Error opening adjoint_psi.txt" << std::endl;
}
else
{
   for (int i = 0; i < N; ++i)
   {
      psi_file << i << " " << psi[i] << "\n";
   }
   psi_file.close();
}
std::vector<double> dR_da(N);
std::vector<double> dR_db(N);
std::vector<double> dR_dc(N);

dR_da[0] = 0;
dR_db[0] = 0;
dR_dc[0] = 0;

for (int i = 1; i < N; ++i)
{
```

```
    double xi   = x[i];
    double di   = delta[i];
    double xmid = x[i] + dx/2;

    dR_da[i] = -2*B*dx*di*(1.0 / ((2*A*di*di + B*dx - dx*di*v0(xi,a,b,c))*(2*A*di*di + B*dx -
dx*di*v0(xi,a,b,c))))*dv0_da(xi,a,b,c)*dx*di - (dx/A)*dv0_da(xmid,a,b,c);
    dR_db[i] = -2*B*dx*di*(1.0 / ((2*A*di*di + B*dx - dx*di*v0(xi,a,b,c))*(2*A*di*di + B*dx -
dx*di*v0(xi,a,b,c))))*dv0_db(xi,a,b,c)*dx*di - (dx/A)*dv0_db(xmid,a,b,c);
    dR_dc[i] = -2*B*dx*di*(1.0 / ((2*A*di*di + B*dx - dx*di*v0(xi,a,b,c))*(2*A*di*di + B*dx -
dx*di*v0(xi,a,b,c))))*dv0_dc(xi,a,b,c)*dx*di - (dx/A)*dv0_dc(xmid,a,b,c);
  }

  double psiT_dR_da = 0.0;
  double psiT_dR_db = 0.0;
  double psiT_dR_dc = 0.0;

  for (int i = 0; i < N; ++i)
  {
    psiT_dR_da += -psi[i] * dR_da[i];
    psiT_dR_db += -psi[i] * dR_db[i];
    psiT_dR_dc += -psi[i] * dR_dc[i];
  }

  std::cout << std::scientific << std::setprecision(12);

  std::cout << "----------------------------\n";
  std::cout << "Discrete Adjoint Gradients\n";
  std::cout << "dF/da (Adjoint) = " << psiT_dR_da << "\n";
  std::cout << "dF/db (Adjoint) = " << psiT_dR_db<< "\n";
  std::cout << "dF/dc (Adjoint) = " << psiT_dR_dc << "\n";

  //ALM
```

```
// ALM params
int outer_max = 20;
int inner_max = 40;

double lambdaALM = -0.01;
double omega = 3.0;
double omega_mult = 1.1;

double eps_c = 1e-8;
double eps_g = 1e-8;
double alpha = 3;

for (int kout = 0; kout < outer_max; ++kout)
{
   // ===== INNER=====
   for (int kin = 0; kin < inner_max; ++kin)
   {

      double dFda, dFdb, dFdc;
      double F = compute_F_and_grad_CA(a, b, c, N, dFda, dFdb, dFdc);



      double dcda, dcdb, dcdc;
      double ccon = compute_c_and_grad(a, b, c, dcda, dcdb, dcdc);
      double g_a = dFda - lambdaALM*dcda + 2.0*omega*ccon*dcda;
      double g_b = dFdb - lambdaALM*dcdb + 2.0*omega*ccon*dcdb;
      double g_c = dFdc - lambdaALM*dcdc + 2.0*omega*ccon*dcdc;

      double gnorm = std::sqrt(g_a*g_a + g_b*g_b + g_c*g_c);
      if (gnorm < eps_g) break;
```

```
        // gradient step
        a -= alpha * g_a;
        b -= alpha * g_b;
        c -= alpha * g_c;
    }

    // ===== OUTER: update lambda
    double dcda, dcdb, dcdc;
    double ccon = compute_c_and_grad(a, b, c, dcda, dcdb, dcdc);

    // ALM update
    lambdaALM = lambdaALM - 2.0 * omega * ccon;
    double dFda_tmp, dFdb_tmp, dFdc_tmp;
    double F_now = compute_F_and_grad_CA(a, b, c, N, dFda_tmp, dFdb_tmp, dFdc_tmp);

    std::cout << "OUTER " << kout
        << "  F=" << std::scientific << F_now
        << "  c=" << ccon
        << "  lambda=" << lambdaALM
        << "  omega=" << omega
        << "  a=" << a << " b=" << b << " c=" << c
        << "\n";

    if (std::abs(ccon) < eps_c) break;

    omega *= omega_mult;
}

// final
double Ffinal = computeF(a, b, c, N, true);
```

```
    double dcda_f, dcdb_f, dcdc_f;
    double c_res = compute_c_and_grad(a, b, c, dcda_f, dcdb_f, dcdc_f);

    std::cout << "FINAL: F=" << Ffinal << "  a=" << a << " b=" << b << " c=" << c
            << "  lambda=" << lambdaALM << "\n";
    std::cout << "FINAL CONSTRAINT residual c_con=" << std::scientific << c_res
        << "  (target: 0)\n";

    return 0;
}
```